3 Numerical methods and comparison protocol

Section 2 established that sufficient supports give a sparse, symmetric positive-definite system $Ax = b$. These properties guide how we solve for the free displacements: direct methods factor the matrix, while iterative methods improve an approximation without forming its inverse. We compare methods on the same supported system. Iterations approach one static equilibrium; they are not steps in physical time.

3.1 Direct and iterative solutions

For a small system, Cholesky factorization uses positive definiteness to write $A = LL^T$, where L is lower triangular. We first solve $Ly = b$ for an intermediate vector y, then $L^T x = y$ for the displacement. Pivoted LU instead factors a row-permuted matrix into lower and upper triangular factors. Both avoid explicitly forming A^{-1}. The small example also evaluates $A^{-1}b$ for illustration only. For larger systems, sparse LU supplies a numerical reference x_{ref} on the identical mesh. It is an *untimed reference*, not a measured competitor in the current iterative timing comparison.

The simpler iterative methods correct the current force imbalance, called the residual. Let x_k be the displacement approximation after k iterations, $r_k = b - Ax_k$, and $P = \text{diag}(A)$ the diagonal matrix containing the diagonal entries of A. Jacobi updates all components from their previous values; damped Jacobi scales that update:

$$x_{k+1} = x_k + \omega P^{-1} r_k. \tag{1}$$

Our comparison uses $\omega = 1$ for Jacobi and $\omega = 0.5$ for damped Jacobi. Gauss–Seidel instead updates components sequentially, using the newest values already available. Positive definiteness guarantees Gauss–Seidel convergence in exact arithmetic, but does not guarantee ordinary Jacobi convergence. Damping can help, although its chosen weight is not a universal convergence guarantee [1].

Steepest descent also follows r_k, but selects the step that minimizes the quadratic energy $\frac{1}{2}x^T Ax - b^T x$ along that direction:

$$x_{k+1} = x_k + \alpha_k r_k, \qquad \alpha_k = \frac{r_k^T r_k}{r_k^T A r_k}. \tag{2}$$

Conjugate gradients (CG) improves on this by retaining information from previous corrections. Its search directions are mutually conjugate with respect to A, meaning $p_i^T A p_j = 0$ for distinct directions p_i, p_j in exact arithmetic. Each step minimizes the same energy over the search space built so far. This makes CG a natural choice for our supported positive-definite matrix [1].

Diagonal preconditioned CG (PCG) uses $z_k = P^{-1} r_k$ to scale the residual before constructing its search direction. The positive diagonal makes this scaling compatible with CG without changing the solution. It need not resolve poor conditioning; any benefit must outweigh setup and application costs. In SciPy the argument `M` receives the inverse action P^{-1}, not P itself [2].

We also include GMRES and MINRES as comparisons, not because the matrix requires a nonsymmetric or indefinite solver. Both seek a small Euclidean residual in a growing space generated from the initial residual by repeated multiplication by A. GMRES constructs and stores an orthogonal basis of that space; MINRES exploits symmetry through a short recurrence. Our unpreconditioned GMRES uses a restart length equal to the number of free unknowns, allowing a full search space before any restart. This full-GMRES choice can become expensive in memory and orthogonalization as the mesh grows [1, 3].

3.2 Common stopping and timing rules

All eight iterative methods receive the same A, b and start from $x_0 = 0$. For nonzero b, we accept a computed solution only when an independently recomputed relative residual satisfies

$$\rho(x) = \frac{\|b - Ax\|_2}{\|b\|_2} < 10^{-8}, \tag{3}$$

where $\|\cdot\|_2$ denotes the Euclidean vector norm. The common budget is 20 000 iterations. One iteration means one complete stationary sweep or one Krylov step; for GMRES we count inner Arnoldi steps, not restart cycles. Counts represent different work per step and do not alone establish speed. If $b = 0$, the supported system has $x = 0$ and we report zero iterations and zero absolute residual without dividing by $\|b\|_2$.

We retain both the solver's return status and the independent acceptance flag. CG and PCG use relative tolerance 10^{-8} and zero absolute tolerance. MINRES uses native relative tolerance zero and a callback that stops when the true residual meets Eq. (3); its native stopping test is not treated as an equivalent criterion. GMRES uses SciPy's legacy callback convention so its budget counts inner steps. Its recorded history contains Arnoldi residual estimates, whereas final acceptance always uses $b - Ax$. Other recorded histories use explicitly computed residuals. Budget exhaustion or breakdown is reported rather than removed from the comparison; the stationary and steepest-descent runs also stop if the relative residual is nonfinite or exceeds 10^{10}. An unsuccessful run's time is not a time to the requested accuracy.

The current benchmark records three timed calls per method and reports the minimum, retaining all three values. Residual histories are collected in a separate run. Each timed call includes iterative setup, such as the diagonal preconditioner or Gauss–Seidel splitting, and required stopping checks. MINRES therefore still computes a true residual at each timed iteration. Assembly, condition estimation, the sparse-LU reference and final independent error checks are outside the timer. These are *setup-inclusive iterative times*, not separately measured setup and solve costs. Software and machine details accompany the data. Small timing differences do not establish a general ranking. Separating setup or factorization from solves would require new measurements.

3.3 Accuracy and validation

Alongside runtime, iteration count and $\rho(x)$, we report agreement with the direct reference through

$$e_{\mathrm{ref}}(x) = \frac{\|x - x_{\mathrm{ref}}\|_2}{\|x_{\mathrm{ref}}\|_2}. \tag{4}$$

For a zero reference we use the absolute difference. We check the reference's residual: it is a numerical solution, not an exact answer. Conditioning, introduced in Section 2, explains why a small force imbalance need not imply an equally small displacement error. Numerical stability is assessed through residuals, reference agreement, termination behaviour and the response to a specified small load perturbation at fixed A. Each perturbation test records its magnitude and direction; one test does not establish worst-case sensitivity.

These checks address the solution of one mesh's equations, not the error introduced by approximating a continuous body with triangles. To investigate that separate discretization error, we refine the mesh while preserving geometry, material, physical loads and support regions, and compare displacements at matching locations or consistently defined observables. Agreement between solvers on one mesh is not evidence of continuum accuracy.

In Sections 4 and 6, we reconstruct the full displacement, verify prescribed values and use Section 2's reactions to check total force and moment balance. Free-row residuals are not support reactions. A separate exact affine-displacement test uses roller supports, not the clamped benchmark conditions. Section 4 now applies this protocol to the square example.

References

[1] Yousef Saad, *Iterative methods for sparse linear systems*, second ed., SIAM, 2003.

[2] SciPy developers, *scipy.sparse.linalg.cg*, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.cg.html`, API documentation.

[3] ———, *scipy.sparse.linalg.gmres*, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.gmres.html`, API documentation.